

```
# utils/config_manager_v2_5.py
```

```
class ConfigManagerV2_5:
```

```
    """
```

```
    Handles loading, validation (against JSON Schema), default-filling,  
    and providing access to the EOTS v2.5 configuration.
```

```
    May internally use Pydantic models if that route is chosen.
```

```
    """
```

```
    def __init__(self, config_path: str, schema_path: str):
```

```
        # Initialize paths, load, validate, and fill defaults
```

```
        pass
```

```
    def load_and_validate_config(self):
```

```
        # Logic to read JSON config, read JSON schema, validate, and apply defaults
```

```
        pass
```

```
    def get_config_root(self) -> dict:
```

```
        # Returns the entire (validated, default-filled) config dictionary
```

```
        pass
```

```
    def get_setting(self, key_path: list[str] | str, default_value: Any = None) -> Any:
```

```
        # Simplified, direct lookup on the processed config data
```

```
        pass
```

```
    def get_project_root(self) -> str | None:
```

```
# Returns the detected project root directory
```

```
pass
```

```
def resolve_path(self, relative_or_absolute_path: str) -> str:
```

```
    # Resolves a path relative to the project root
```

```
    pass
```

```
def get_resolved_path_setting(self, key_path: list[str] | str, default_relative_path: str |  
None = None) -> str | None:
```

```
    # Gets a path string from config and resolves it
```

```
    pass
```

```
# End of config_manager_v2_5.py
```

```
# data_management/fetcher_convexvalue_v2_5.py
```

```
class ConvexValueDataFetcherV2_5:
```

```
    """
```

```
    Handles data fetching specifically from the ConvexValue API.
```

```
    (Evolved from v2.4 fetcher.py)
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Initialize API client, credentials, retry settings
```

```
        pass
```

```
    def connect_api(self):
```

```
        # Establish connection to ConvexValue
```

```
        pass
```

```
    def fetch_underlying_data(self, symbol: str, api_params_list: list[str]) -> dict | None:
```

```
        # Fetches get_und data
```

```
        pass
```

```
    def fetch_options_chain_data(self, symbol: str, dte_list: list[int] | None, price_range_pct: float | None, api_params_list: list[str]) -> list[list[Any]] | None:
```

```
        # Fetches get_chain_as_rows data
```

```
        pass
```

```
    def shutdown(self):
```

```
# Cleanup resources if any
```

```
pass
```

```
# End of fetcher_convexvalue_v2_5.py
```

```
# data_management/fetcher_tradier_v2_5.py
```

```
class TradierDataFetcherV2_5:
```

```
    """
```

```
    Handles data fetching specifically from the Tradier API.
```

```
    (Evolved from v2.4 tradier_data_fetcher.py)
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Initialize API client, token, retry settings
```

```
        pass
```

```
    def get_underlying_quote(self, symbol: str) -> dict | None:
```

```
        # Fetches current quote (for OHLCV snapshot)
```

```
        pass
```

```
    def get_historical_ohlc(self, symbol: str, interval: str, start_date: str, end_date: str) ->  
list[dict] | None:
```

```
        # Fetches historical bars
```

```
        pass
```

```
    def get_option_expirations(self, symbol: str) -> list[str] | None:
```

```
        # Fetches expiration dates
```

```
        pass
```

```
def get_option_chain_for_expiration(self, symbol: str, expiration_date: str) -> list[dict] |  
None:
```

```
    # Fetches option chain for a specific expiration
```

```
    pass
```

```
def get_iv_approximation_tradier(self, symbol: str, target_dte: int) -> dict | None:
```

```
    # Placeholder for fetching data that can approximate specific IV (like IV5 using  
SMV_VOL)
```

```
    pass
```

```
def shutdown(self):
```

```
    # Cleanup resources if any
```

```
    pass
```

```
# End of fetcher_tradier_v2_5.py
```

```
# data_management/historical_data_manager_v2_5.py
```

```
class HistoricalDataManagerV2_5:
```

```
    """
```

```
    Manages storage and retrieval of historical OHLCV, IV, and
```

```
    key V2.5 aggregate metrics for dynamic thresholding and ATR calculations.
```

```
    (Evolved from v2.4)
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Initialize storage paths, metrics to track
```

```
        pass
```

```
    def store_daily_ohlcw(self, symbol: str, data_date: 'date', ohlcw_data: dict):
```

```
        # Stores daily OHLCV bar
```

```
        pass
```

```
    def get_ohlcw_history_for_atr(self, symbol: str, num_days: int, current_trading_date:
'date') -> 'pd.DataFrame' | None:
```

```
        # Retrieves OHLCV for ATR calculation
```

```
        pass
```

```
    def store_daily_aggregate_metric(self, symbol: str | None, data_date: 'date', metric_key:
str, value: Any):
```

```
        # Stores a calculated V2.5 aggregate metric value for a symbol or general market
```

```
        pass
```

```
def get_metric_distribution(self, symbol: str | None, metric_key: str, days_history: int,
current_trading_date: 'date') -> 'pd.Series' | None:
```

```
    # Retrieves historical values for a specific aggregate metric for dynamic thresholding
```

```
    pass
```

```
def get_historical_iv_for_trend(self, symbol: str, period_days: int, current_date: 'date') ->
'pd.Series' | None:
```

```
    # Retrieves a series of historical IV (could be from specific stored metric like
"tradier_iv5_approx_smv_avg")
```

```
    pass
```

```
def shutdown(self):
```

```
    # Cleanup resources if any
```

```
    pass
```

```
# End of historical_data_manager_v2_5.py
```



```
# data_management/performance_tracker_v2_5.py
```

```
class PerformanceTrackerV2_5:
```

```
    """
```

```
    [NEW] Stores, retrieves, and analyzes historical performance data  
    for signals, recommendations, and strategy parameters under different  
    market regimes. Feeds data to the Adaptive Trade Idea Framework.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Initialize storage paths, performance metrics to track
```

```
        pass
```

```
    def record_recommendation_outcome(self, recommendation_id: str, outcome_data:  
dict):
```

```
        # Records the result of a closed/exited trade recommendation
```

```
        # Outcome data includes P&L, exit reason, regime at exit, key metric values at exit, etc.
```

```
        pass
```

```
    def get_signal_performance_summary(self, signal_name_pattern: str, market_regime: str  
| None = None) -> dict:
```

```
        # Calculates historical success rate, avg P&L, etc., for specific signals or patterns
```

```
        # optionally filtered by market regime. Used by ATIF for dynamic weighting.
```

```
        pass
```

```
    def get_strategy_parameter_effectiveness(self, strategy_type: str, parameter_name: str) -  
> dict:
```

```
    # Analyzes effectiveness of different parameter values for strategies (e.g., ATR  
multipliers for stops)
```

```
    # More advanced, for future self-tuning of TPO.
```

```
    pass
```

```
def shutdown(self):
```

```
    # Cleanup resources if any
```

```
    pass
```

```
# End of performance_tracker_v2_5.py
```

```
# data_management/initial_processor_v2_5.py
```

```
class InitialDataProcessorV2_5:
```

```
    """
```

```
    (Evolved from v2.4)
```

```
    Validates, cleans, and prepares the raw data bundle from fetchers.
```

```
    Passes prepared data to MetricsCalculatorV2_5.
```

```
    Packages final results including all calculated metrics.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5', metrics_calculator:  
'MetricsCalculatorV2_5'):
```

```
        # Initialize column names, default values
```

```
        pass
```

```
    def process_snapshot_data(self, raw_data_bundle_from_fetchers: dict, current_time_dt:  
'datetime', symbol: str) -> dict:
```

```
        # 1. Validate and prepare raw options_df and underlying_dict
```

```
        # 2. Invoke metrics_calculator.orchestrate_all_metric_calculations_v2_5
```

```
        # 3. Package inputs, metrics, and any errors into a final bundle
```

```
        pass
```

```
# End of initial_processor_v2_5.py
```

```
# core_analytics_engine/metrics_calculator_v2_5.py
```

```
class MetricsCalculatorV2_5:
```

```
    """
```

```
    [NEW or Heavily Refactored from v2.4]
```

```
    Calculates ALL system metrics:
```

- Base V2.4 metrics (GIB, NVP, basic Rolling Flows, etc.)
- New Enhanced Rolling Flow Metrics (VAPI-FA, DWFD, TW-LAF)
- New Adaptive Metrics (A-DAG, E-SDAG, D-TDPI, VRI 2.0)
- Data arrays for new Enhanced Heatmaps (SGDHP, IVSDH, UGCH data calculation, not plotting)
- Interacts with HistoricalDataManager for ATR, historical IV, etc.
- Receives context from SPYSPXOptimizer for SPY/SPX specific adjustments.

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5', historical_data_manager:  
'HistoricalDataManagerV2_5'):
```

```
        # Initialize calculation parameters, column names from config
```

```
        pass
```

```
    def orchestrate_all_metric_calculations_v2_5(self,
```

```
        options_df_prepared: 'pd.DataFrame',
```

```
        underlying_data_prepared: dict,
```

```
        current_time_dt: 'datetime',
```

```
        symbol: str,
```

```

        spyspx_context: dict | None = None) -> tuple['pd.DataFrame',
'pd.DataFrame', dict]:

    # Main orchestration method

    # 1. Set instance variables (current_under_data, current_price, etc.)

    # 2. Calculate common context columns (DTE, etc.) on options_df

    # 3. Call individual calculation methods in logical order

    # - Base underlying aggregates (GIB, td_gib, NVP (from contract sums), Basic Rolling
    Flows (from contract sums))

    # - Per-contract Adaptive Metrics (A-DAG components, E-SDAG components, D-TDPI
    components, VRI 2.0 components)

    # - Per-contract Enhanced Flow Metrics (VAPI-FA components, DWFD components,
    TW-LAF components from base rolling flows)

    # - Aggregate per-contract to strike-level (df_strike_level_metrics)

    # - Calculate MSPI, SAI, SSI using adaptive components on strike-level data

    # - Calculate final underlying aggregates (HP_EOD, aggregate flow metrics, aggregate
    heatmap data if needed for MRE)

    # Returns:

    # - options_df_with_all_contract_metrics

    # - df_strike_level_all_metrics

    # - underlying_data_enriched_with_all_aggregates

    pass

# --- Placeholder for individual metric calculation groups ---

# (Each of these would have multiple private methods)

def _calculate_base_v2_4_metrics_group(self, options_df: 'pd.DataFrame',
underlying_data: dict) -> tuple['pd.DataFrame', dict]:

```

```
# GIB_OI_based_Und, td_gib_Und, HP_EOD_Und (needs GIB), NVP_strike,  
Basic_Rolling_Flows_Und
```

```
pass
```

```
def _calculate_adaptive_metrics_group(self, options_df: 'pd.DataFrame',  
underlying_data: dict, spyspx_context: dict | None) -> 'pd.DataFrame':
```

```
# A-DAG, E-SDAG, D-TDPI, VRI 2.0 (these are complex, involving many sub-steps)
```

```
# These methods will take the output from _calculate_base_v2_4_metrics_group
```

```
# and the current market regime (passed via underlying_data or as a direct param)
```

```
# and SPY/SPX context.
```

```
pass
```

```
def _calculate_enhanced_rolling_flow_metrics_group(self, options_df: 'pd.DataFrame',  
underlying_data: dict) -> dict:
```

```
# VAPI-FA, DWFD, TW-LAF (calculated at underlying level, using base rolling flows from  
options_df and other metrics)
```

```
pass
```

```
def _calculate_heatmap_data_arrays_group(self, df_strike_level_metrics:  
'pd.DataFrame', underlying_data: dict) -> dict:
```

```
# Calculate and store data arrays/dictionaries needed for SGDHP, IVSDH, UGCH  
visualizations
```

```
# These might be stored in underlying_data or returned separately.
```

```
pass
```

```
# --- Utility methods (ATR, normalization etc. might be here or in utils) ---
```

```
def _get_atr_for_metrics(self, symbol: str, current_price: float, current_trading_date:  
'date') -> float:
```

pass

End of metrics_calculator_v2_5.py

```
# core_analytics_engine/market_regime_engine_v2_5.py
```

```
class MarketRegimeEngineV2_5:
```

```
    """
```

```
    (Evolved from v2.4)
```

```
    Classifies the current market regime using a richer set of V2.5 metric inputs
```

```
    (adaptive metrics, new flow metrics) and SPY/SPX contextual information.
```

```
    Rules are defined in config_v2_5.json.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Initialize regime rules, evaluation order, time definitions, dynamic threshold logic
```

```
        pass
```

```
    def determine_market_regime_v2_5(self,
```

```
        und_data_aggregates_enriched: dict, # Contains all V2.5 aggregate  
metrics
```

```
        df_strike_level_metrics_v2_5: 'pd.DataFrame' | None,
```

```
        current_time_dt: 'datetime',
```

```
        resolved_dynamic_thresholds: dict,
```

```
        spyspx_context: dict | None = None, # New input
```

```
        symbol: str | None = None) -> str:
```

```
        # 1. Access V2.5 metrics from inputs
```

```
        # 2. Incorporate spyspx_context (e.g., "IS_ODTE_FRIDAY_PM", "IS_FOMC_DAY") into  
condition evaluation
```

```
        # 3. Evaluate richer regime_rules from config_v2_5.json
```

```
        # Returns the classified regime string
```


pass

End of market_regime_engine_v2_5.py

```
# core_analytics_engine/spyspx_optimizer_v2_5.py
```

```
class SPYSPXOptimizerV2_5:
```

```
    """
```

```
    [NEW or Integrated Logic]
```

```
    Handles SPY/SPX specific logic:
```

- Expiration calendar awareness (0DTE, M/W/F, EOM, Triple Witching).
- Behavior pattern recognition (FOMC, earnings impact on components, VIX divergence).
- Intraday pattern adjustments (opening, midday, power hour, auctions).

```
    Outputs contextual flags/parameters to other EOTS components.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Load SPY/SPX specific configs (e.g., known pattern definitions, component weights)
```

```
        pass
```

```
    def get_current_spyspx_context(self, current_time_dt: 'datetime', underlying_data: dict,  
options_data_summary: dict | None) -> dict:
```

```
        # Determines:
```

```
        # - Current expiration type characteristics (is_0DTE, is_M_W_F_expiry,  
days_to_next_major_expiry)
```

```
        # - Active intraday period (e.g., "OPENING_VOL", "LUNCH_LULL", "POWER_HOUR")
```

```
        # - Recognized behavior patterns (e.g., "FOMC_PRE_ANNOUNCEMENT",  
"TECH_SECTOR_LEADING")
```

```
        # Returns a dictionary of context flags and relevant data.
```

```
        pass
```

```
# End of spyspx_optimizer_v2_5.py
```

```
# core_analytics_engine/signal_generator_v2_5.py
```

```
class SignalGeneratorV2_5:
```

```
    """
```

```
    (Evolved from v2.4)
```

```
    Generates more nuanced, potentially scored trading signals based on V2.5 adaptive metrics,
```

```
    enhanced flow metrics, market regime, and SPY/SPX context.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Load signal activation flags, thresholds (many will be dynamic via Orchestrator)
```

```
        pass
```

```
    def generate_all_signals_v2_5(self,
```

```
        df_metrics_strike_level_v2_5: 'pd.DataFrame',
```

```
        und_data_aggregates_enriched_v2_5: dict,
```

```
        current_market_regime_v2_5: str,
```

```
        current_time_dt: 'datetime',
```

```
        resolved_dynamic_thresholds: dict,
```

```
        spyspx_context_v2_5: dict | None) -> dict[str, dict[str, list[dict]]]:
```

```
        # 1. Generate signals using V2.5 metrics (A-DAG, VAPI-FA, SGDHP data etc.)
```

```
        # 2. Signals might output a continuous score for conviction.
```

```
        # 3. Incorporate spyspx_context to modulate signal strength or type.
```

```
        # Returns structured dictionary of signals.
```

pass

End of signal_generator_v2_5.py

```
# core_analytics_engine/adaptive_trade_idea_framework_v2_5.py
```

```
class AdaptiveTradeIdeaFrameworkV2_5:
```

```
    """
```

```
    [NEW - CENTRAL BRAIN] Replaces/significantly augments v2.4  
    recommendation_logic.py.
```

- Dynamic Signal Integration (performance-based weighting, regime context).
- Performance-Based Conviction Mapping.
- Enhanced Strategy Specificity (recommends specific option contracts/structures).
- Intelligent Recommendation Management (outputs directives for exits, adjustments).

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5', orchestrator_ref:  
'ITSOrchestratorV2_5', performance_tracker: 'PerformanceTrackerV2_5'):
```

```
        # Load ATIF specific configurations (learning rates, strategy selection rules, etc.)
```

```
        # Store reference to orchestrator for helper methods (e.g., map_score_to_stars IF ATIF  
        doesn't do its own full mapping)
```

```
        # Store reference to performance_tracker
```

```
        pass
```

```
    def generate_trade_recommendations_v2_5(self,
```

```
        scored_signals_from_sg: dict, # From signal_generator_v2_5
```

```
        current_market_regime: str,
```

```
        spyspx_context: dict | None,
```

```
        current_und_price: float,
```

```
        available_options_data: 'pd.DataFrame', # To select actual contracts
```

```
        key_levels_data: dict, # From key_level_identifier_v2_5
```

```

        current_time_dt: 'datetime',

        symbol: str) -> list[dict]: # List of fully formed recommendation
payloads (PENDING_PARAMETERS status)

    # 1. Integrate scored signals dynamically (using performance weights from tracker,
regime context)

    # 2. Map integrated assessment to overall conviction (using performance-based
mapping)

    # 3. Select optimal option strategy type (long call, spread, iron condor etc.) based on
signals, regime, IV (from und_data), SPY/SPX context.

    # 4. Determine target DTE, delta range for selected strategy.

    # 5. Construct initial recommendation payload (status: PENDING_PARAMETERS).

    # Payload includes all context for TPO.

    pass

```

```

def get_management_directives_for_active_recommendation(self,

        active_recommendation_payload: dict,

        current_market_data_bundle: dict, # All current metrics,
regime, etc.

        spyspx_context: dict | None) -> dict | None:

    # Logic for adaptive exits (SL adjustments, target adjustments, full exits)

    # Logic for partial position management (scale in/out based on evolving VAPI-FA,
DWFD, regime changes)

    # Returns a directive: e.g., {"action": "ADJUST_SL", "new_sl": X}, {"action": "EXIT",
"reason": "XYZ"}

    pass

```

```

# End of adaptive_trade_idea_framework_v2_5.py

```

```
# core_analytics_engine/trade_parameter_optimizer_v2_5.py
```

```
class TradeParameterOptimizerV2_5:
```

```
    """
```

```
    (Evolved from v2.4)
```

```
    Receives a recommendation payload (with strategy type, entry, bias, context) from ATIF.
```

```
    Calculates precise entry suggestions (if needed), stop-losses, and profit targets.
```

```
    Uses Enhanced Key Levels (from key_level_identifier_v2_5), VRI 2.0 for dynamic ATR,  
    and regime-specific multipliers.
```

```
    Selects specific option strikes from available_options_data based on delta targets from  
    ATIF.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5', orchestrator_ref:  
    'ITSOrchestratorV2_5'):
```

```
        # Load target/stop calculation parameters, S/R source preferences
```

```
        pass
```

```
    def optimize_and_select_contract_parameters(self,
```

```
        pending_recommendation_payload: dict, # From ATIF
```

```
        df_strike_level_metrics_v2_5: 'pd.DataFrame',
```

```
        und_data_aggregates_enriched_v2_5: dict,
```

```
        key_levels_data_v2_5: dict, # From KeyLevelIdentifier
```

```
        available_options_data_for_selection: 'pd.DataFrame', # Full chain  
    for selection
```

```
        symbol: str,
```



```
current_time_dt: 'datetime') -> dict: # Returns updated  
recommendation payload
```

```
# 1. Get ATR (using VRI 2.0 context if available for multiplier)
```

```
# 2. Identify relevant S/R levels from key_levels_data_v2_5 (using SGDHP, UGCH, NVP  
sources)
```

```
# 3. Based on strategy_type, DTE_target, delta_target from ATIF:
```

```
# - Select best fit option contract(s) from available_options_data_for_selection.
```

```
# - Calculate precise entry (e.g., mid-price of selected contract).
```

```
# 4. Calculate stop-loss and profit targets for the selected contract(s).
```

```
# 5. Populate the recommendation payload with these parameters and  
"ACTIVE_NEW_NO_TSL" status.
```

```
pass
```

```
# End of trade_parameter_optimizer_v2_5.py
```

```
# core_analytics_engine/key_level_identifier_v2_5.py
```

```
class KeyLevelIdentifierV2_5:
```

```
    """
```

```
    [NEW or Integrated Logic]
```

```
    Implements the Enhanced Key Level Identification framework:
```

- Multi-timeframe analysis (if historical MSPI/A-MSPI aggregates are stored).
- Advanced Wall/Trigger detection using GIB, and data from new heatmaps (SGDHP, UGCH).
- Conviction-based level scoring based on metric alignment and historical interaction (if tracked).

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5', historical_data_manager:  
'HistoricalDataManagerV2_5' | None = None):
```

```
        # Load parameters for level identification, strength scoring
```

```
        pass
```

```
    def identify_all_key_levels_v2_5(self,  
  
        df_strike_level_metrics_v2_5: 'pd.DataFrame', # Contains A-MSPI, NVP,  
etc.  
  
        heatmap_data_arrays_v2_5: dict, # Data from SGDHP, UGCH  
calculations  
  
        underlying_data_aggregates_v2_5: dict, # For GIB, current price  
  
        current_price: float) -> dict: # Output: {"supports": [...], "resistances": [...],  
"vol_triggers": [...]}  
  
        # 1. Identify S/R from A-MSPI (multi-timeframe if applicable).  
  
        # 2. Identify S/R from NVP peaks.
```

3. Identify S/R from heatmap data (strong SGDHP, UGCH levels).

4. Identify Volatility Triggers (e.g., from E-SDAG Volatility-Focused, IVSDH data).

5. Score levels based on confluence, historical interaction (if performance_tracker provides this).

Returns a structured dictionary of identified key levels with their types and conviction scores.

pass

End of key_level_identifier_v2_5.py

```
# core_analytics_engine/its_orchestrator_v2_5.py
```

```
class ITSOrchestratorV2_5:
```

```
    """
```

```
    (Significantly Evolved from v2.4)
```

```
    Manages the main EOTS v2.5 analysis cycle.
```

```
    Integrates all core components, including the new ATIF.
```

```
    Handles stateful management of recommendations based on ATIF directives.
```

```
    """
```

```
    def __init__(self, config_manager: 'ConfigManagerV2_5'):
```

```
        # Dynamically load and initialize ALL V2.5 components:
```

```
        # - Fetchers (ConvexValue, Tradier)
```

```
        # - HistoricalDataManager, PerformanceTracker
```

```
        # - InitialProcessor
```

```
        # - MetricsCalculatorV2_5
```

```
        # - SPYSPXOptimizerV2_5
```

```
        # - MarketRegimeEngineV2_5
```

```
        # - SignalGeneratorV2_5
```

```
        # - KeyLevelIdentifierV2_5
```

```
        # - AdaptiveTradeIdeaFrameworkV2_5 (passing self and PerformanceTracker)
```

```
        # - TradeParameterOptimizerV2_5 (passing self)
```

```
        # Initialize state variables (active_recommendations, history dequeues, etc.)
```

```
        pass
```

```

def fetch_data_for_analysis_cycle(self, symbol: str, dte_list: list[int] | None,
price_range_pct: float | None) -> dict:

    # Combines fetching from ConvexValue and Tradier (for OHLCV to pass to
InitialProcessor,

    # and potentially for supplementary IV if ConvexValue doesn't have what's needed for
MetricsCalculator)

    # Returns a bundle: {"raw_options_df": ..., "raw_underlying_dict_combined": ..., "error":
...}

    pass

```

```

def run_analysis_cycle_v2_5(self, symbol: str, raw_data_bundle_from_fetchers: dict,
current_processing_datetime: 'datetime') -> dict:

```

```

    # 1. Handle symbol change: reset caches, active recommendations.

    # 2. Call InitialProcessorV2_5 -> outputs prepared_options_df,
prepared_underlying_dict (which includes calculated metrics).

    # The 'prepared_underlying_dict' is the result of
metrics_calculator_v2_5.orchestrate...

    # (This means InitialProcessor now *contains* the call to metrics_calculator)

    # Let's refine: Fetch -> InitialProcess (basic prep) -> MetricsCalc (all metrics) ->
SPYSPXContext -> MRE -> Signals -> KeyLevels -> ATIF -> TPO -> ManageState

```

```

# REFINED WORKFLOW STEP FOR ORCHESTRATOR:

# A. FETCH (done by caller, `run_system_dashboard.py`)

# B. INITIAL PROCESSING (by self.initial_data_processor) - this now *includes* metrics
calculation

# processed_bundle = self.initial_data_processor.process_snapshot_data(

#         raw_data_bundle_from_fetchers["raw_options_df"],

#         raw_data_bundle_from_fetchers["raw_underlying_dict_combined"],

#         current_processing_datetime,

```

```

#             symbol
#         )

# df_chain_metrics = processed_bundle["options_df_with_metrics_obj"]
# df_strike_metrics = processed_bundle["df_strike_level_metrics_obj"]
# und_data_enriched = processed_bundle["underlying_data_enriched_obj"]

# C. SPY/SPX CONTEXT (self.spyspx_optimizer)

# spyspx_current_context = self.spyspx_optimizer.get_current_spyspx_context(...)

# D. MARKET REGIME (self.market_regime_engine)

# current_regime =
self.market_regime_engine.determine_market_regime_v2_5(und_data_enriched,
df_strike_metrics, ..., spyspx_current_context)

# E. SIGNAL GENERATION (self.signal_generator)

# scored_signals = self.signal_generator.generate_all_signals_v2_5(df_strike_metrics,
und_data_enriched, current_regime, ..., spyspx_current_context)

# F. KEY LEVEL IDENTIFICATION (self.key_level_identifier)

# key_levels_info =
self.key_level_identifier.identify_all_key_levels_v2_5(df_strike_metrics,
heatmap_data_from_metrics_calc, und_data_enriched, current_price)

# G. ADAPTIVE TRADE IDEA FRAMEWORK (self.atif)

# pending_recos_from_atif =
self.atif.generate_trade_recommendations_v2_5(scored_signals, current_regime,
spyspx_current_context, current_price, df_chain_metrics, key_levels_info, ...)

# H. TRADE PARAMETER OPTIMIZATION (self.trade_parameter_optimizer)

# parameterized_new_recos =
[self.tpo.optimize_and_select_contract_parameters(reco, df_strike_metrics,
und_data_enriched, key_levels_info, df_chain_metrics, ...) for reco in
pending_recos_from_atif]

# I. STATEFUL MANAGEMENT (using ATIF for directives)

```

```
# Update self.active_recommendations based on ATIF directives for existing recos,  
and add new parameterized_recos.
```

```
# Record outcomes to self.performance_tracker.
```

```
# J. STORE HISTORICAL DATA (self.historical_data_manager) - Store key V2.5 aggregate  
metrics.
```

```
# K. PACKAGE & RETURN FINAL ANALYSIS BUNDLE (for dashboard)
```

```
pass
```

```
def _manage_active_recommendations_with_atif_v2_5(self,  
current_market_data_bundle_for_mgmt: dict, spyspx_context_for_mgmt: dict | None):
```

```
# Iterate active_recommendations. For each, call  
self.atif.get_management_directives...
```

```
# Apply directives (update status, SL, TPs, partials, exit). Record to  
performance_tracker if exited.
```

```
pass
```

```
def shutdown_cleanup(self):
```

```
# Call shutdown on all components
```

```
pass
```

```
# End of its_orchestrator_v2_5.py
```